DM4ML Lab Assignment 1

Batch Ingestion and OLAP Operations with Apache Druid

Course	Data Management for Machine Learning (DM4ML)
Roll No	2025AG05637
Druid datasource	dm4ml_2025AG05637
Dataset	Global Superstore - Kaggle fatihilhan/global-superstore-dataset
Tools	Kaggle API, Python/pandas (Jupyter), Apache Druid (micro-quickstart, WSL), Druid SQL
Report generated	2026-10-27

1. Dataset selection and download (Kaggle API)

The **Global Superstore** dataset contains retail order lines from a global store between 2011 and 2014. It fits the brief: it is medium sized, has several dimensions (order date, market, region, country, segment, category, sub-category, ship mode, order priority) and several numeric metrics (sales, quantity, discount, profit, shipping cost).

Item	Value
Kaggle dataset	fatihilhan/global-superstore-dataset
Downloaded file	superstore.csv
Raw shape	51,290 rows x 27 columns
Clean shape	50,425 rows x 29 columns
Date range	2011-01-01 to 2014-12-31

The notebook authenticates with `KaggleApi().authenticate()` using the API token stored in `~/.kaggle/access_token` (never in the notebook), lists the dataset files and downloads them with `dataset_download_files(..., unzip=True)`, then loads the CSV into a pandas DataFrame.

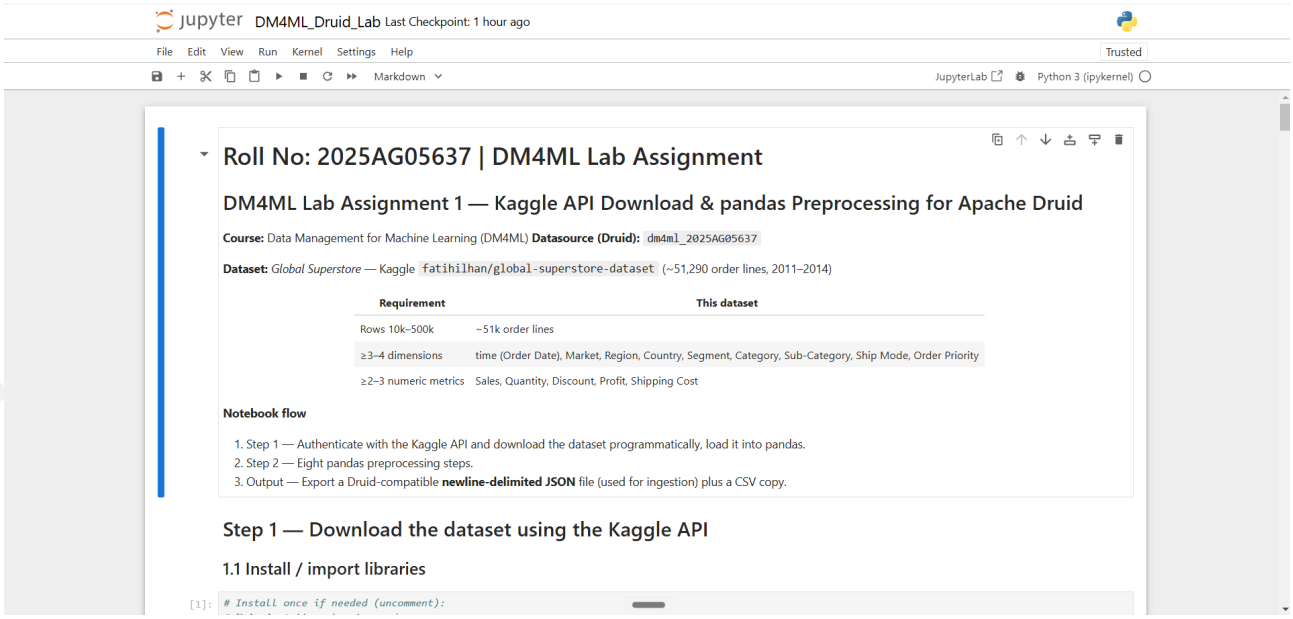


Figure: Jupyter: Kaggle API authentication and dataset download

2. Data cleaning and preprocessing (pandas)

#	Step	What was done	Rows after
1	Standardise columns	Kaggle headers like Order.Date converted to snake_case; constant helper column and Row.ID dropped.	51,290
2	Trim text	Whitespace stripped and empty strings turned into missing values.	51,290
3	Convert types	order_date/ship_date to datetime; metrics to numeric; dimensions to category dtype.	51,290
4	Missing values	Rows missing critical fields dropped; discount/shipping cost filled with 0; categorical gaps set to Unknown. (dropped 0 rows)	51,290
5	Duplicates	Exact duplicate rows removed. (removed 0 (found 0))	51,290
6	Feature engineering	order_year, order_quarter, order_month, month name, day of week, ship_days.	51,290
7	Derived metrics	unit_price, gross_sales, discount_amount, cost, profit_margin.	51,290
8	Invalid records and outliers	quantity/sales <= 0, bad discounts, ship_days outside 0-30, and unit prices above Q3 + 3xIQR per sub-category removed. (invalid=1, price outliers=864)	50,425

Output: `data/superstore_clean.json` - newline-delimited JSON (one object per line), validated in the notebook by parsing every line and matching the row count; a CSV copy is also written.

No: 2025AG05637 | DM4ML Lab Assignm

3. Apache Druid installation and batch ingestion

Apache Druid was downloaded from the official Apache Druid website, extracted inside WSL (Ubuntu) and started with the single-node `./bin/start-micro-quickstart` configuration. The ingestion specification below was submitted to the Overlord through the router (`POST /druid/indexer/v1/task`, script `submit_ingestion.sh`) and verified in the web console at http://localhost:8888.

Setting	Value
Task type	index_parallel (native batch)
Datasource	dm4ml_2025AG05637
Timestamp	order_date (format yyyy-MM-dd)
Dimensions	market, region, country, segment, category, sub_category, ship_mode, order_priority, order_year, order_quarter, order_month
Metrics	line_count (count), sales (doubleSum), quantity (longSum), profit (doubleSum), shipping_cost (doubleSum), gross_sales (doubleSum), discount_amount (doubleSum), cost (doubleSum), ship_days (longSum), max_unit_price (doubleMax), min_unit_price (doubleMin)
Segment / query granularity	month / day, rollup=True
Input	local: /mnt/c/DM4ML_Druid_Lab/data/superstore_clean.json
Input format	json (newline-delimited JSON)

Ingestion task: index_parallel_dm4ml_2025AG05637_fhkomfca_2026-10-27T17:11:42.089Z - status **SUCCESS**, duration 70477 ms.

Datasource check (Druid SQL): `[{"stored_rows":46631,"order_lines":50425}]` - stored_rows is the number of rows after rollup; order_lines is the number of input rows they represent.

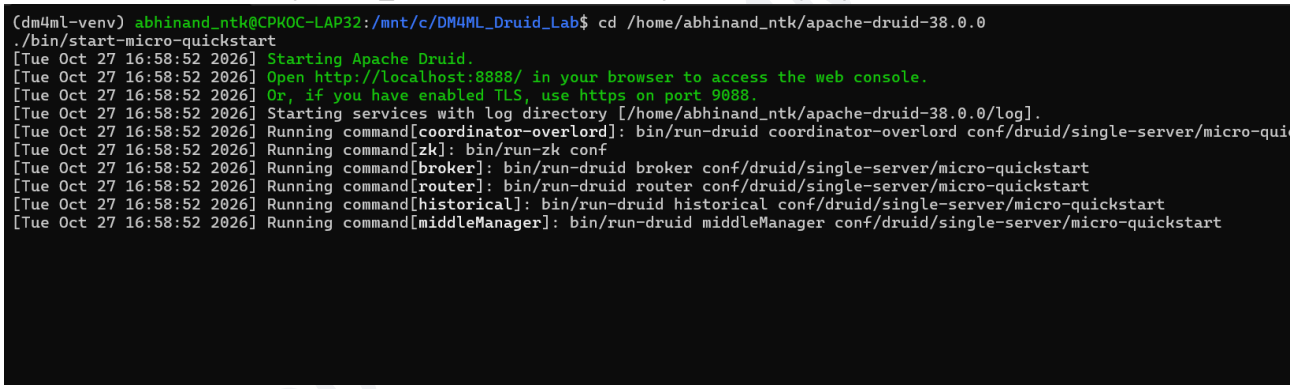


Figure: WSL terminal: ./bin/start-micro-quickstart running

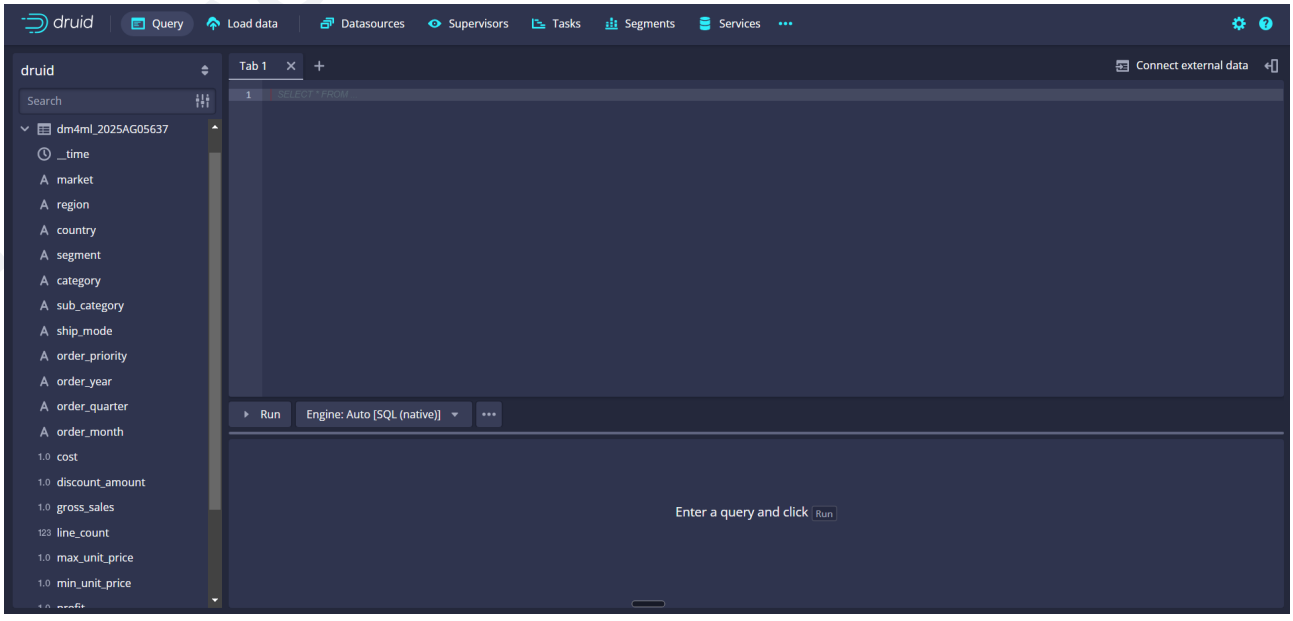


Figure: Druid web console home at http://localhost:8888

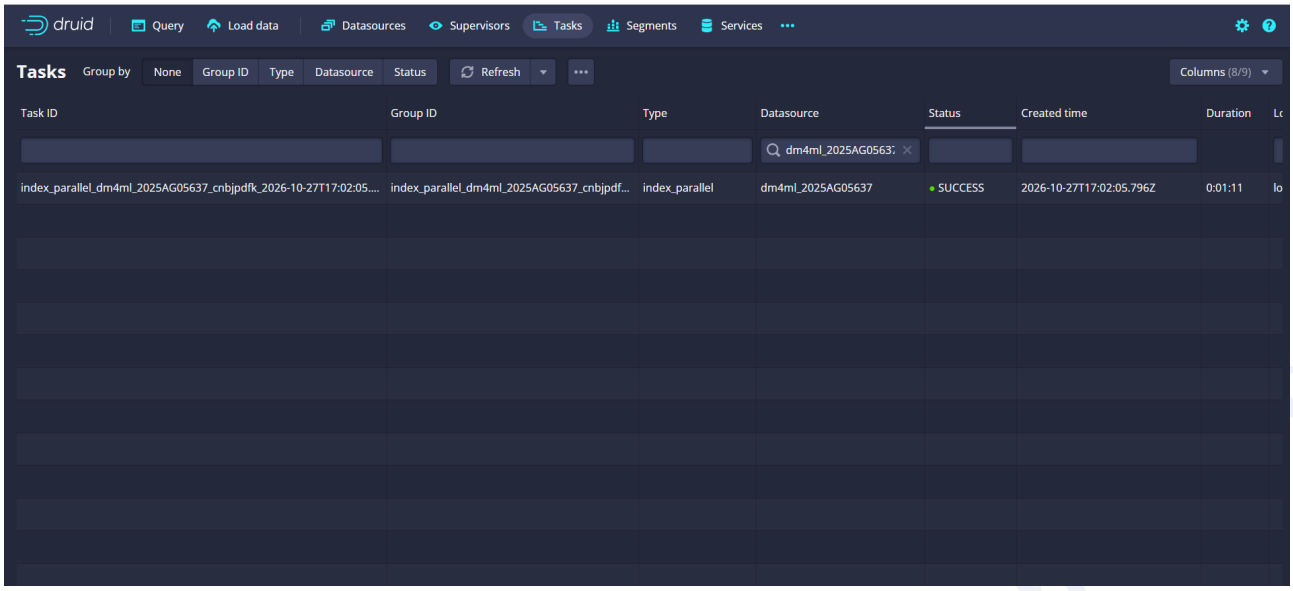


Figure: Tasks view: batch ingestion task with status SUCCESS

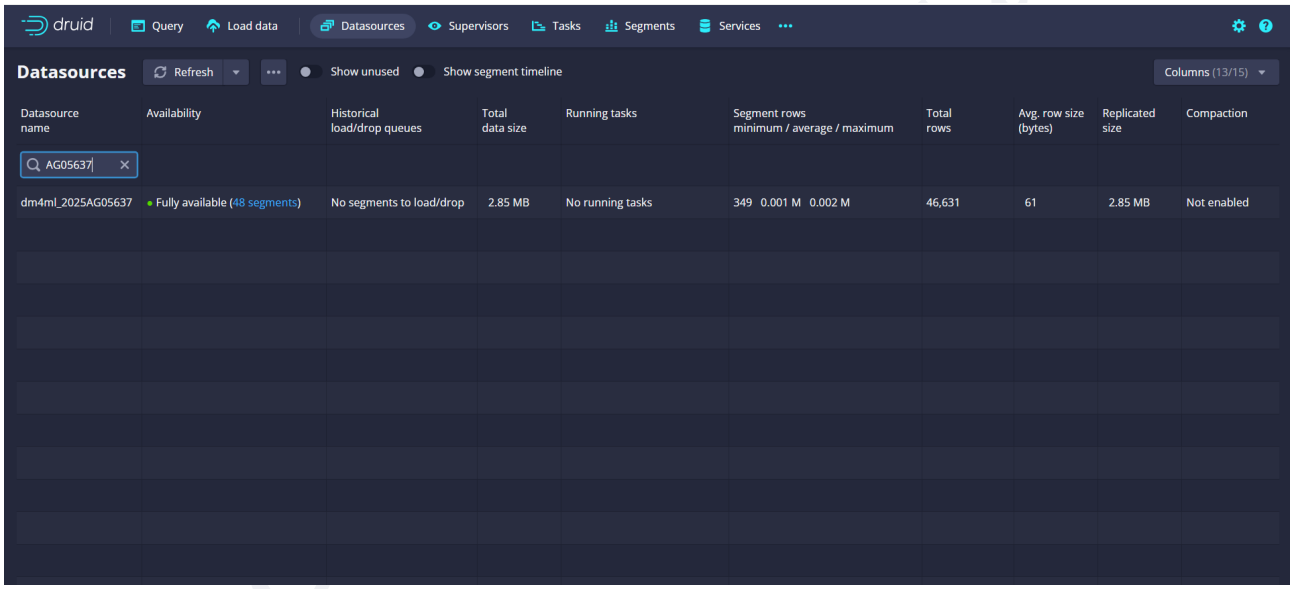


Figure: Datasources view: dm4ml_2025AG05637 fully available

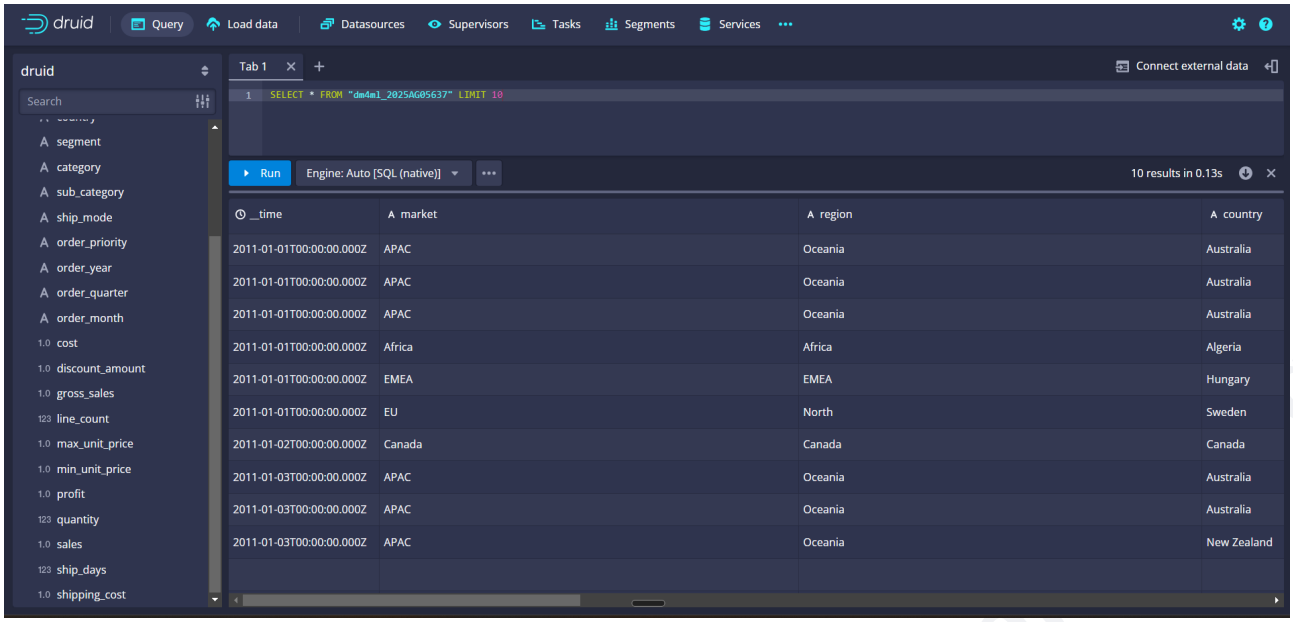


Figure: Query view: SELECT * FROM "dm4ml_2025AG05637" LIMIT 10

No: 2025AG05637 | DM4ML Lab Assignm

4. OLAP vs OLTP demonstration

DELETE and UPDATE statements from `oltp_test.sql` were executed with Druid SQL against `dm4ml_2025AG05637`. Druid's exact responses are reproduced below.

T0 - HTTP 200

```
SELECT COUNT(*) AS stored_rows, SUM(line_count) AS order_lines, ROUND(SUM(sales), 2) AS total_sales
FROM "dm4ml_2025AG05637"
```

Response: [{"stored_rows": 46631, "order_lines": 50425, "total_sales": 11231790.0}]

T1 - HTTP 400

```
DELETE FROM "dm4ml_2025AG05637"
WHERE category = 'Furniture' AND market = 'Canada'
```

Response: [invalidInput] Unsupported SQL statement [DELETE]

T2 - HTTP 400

```
UPDATE "dm4ml_2025AG05637"
SET sales = sales * 1.10
WHERE category = 'Technology' AND __time >= TIMESTAMP '2014-01-01 00:00:00'
```

Response: [invalidInput] Unsupported SQL statement [UPDATE]

T3 - HTTP 200

```
SELECT COUNT(*) AS stored_rows, SUM(line_count) AS order_lines, ROUND(SUM(sales), 2) AS total_sales
FROM "dm4ml_2025AG05637"
```

Response: [{"stored_rows": 46631, "order_lines": 50425, "total_sales": 11231790.0}]

Observation: the counts before (T0) and after (T3) the attempts are identical - the statements had no effect on the stored data.

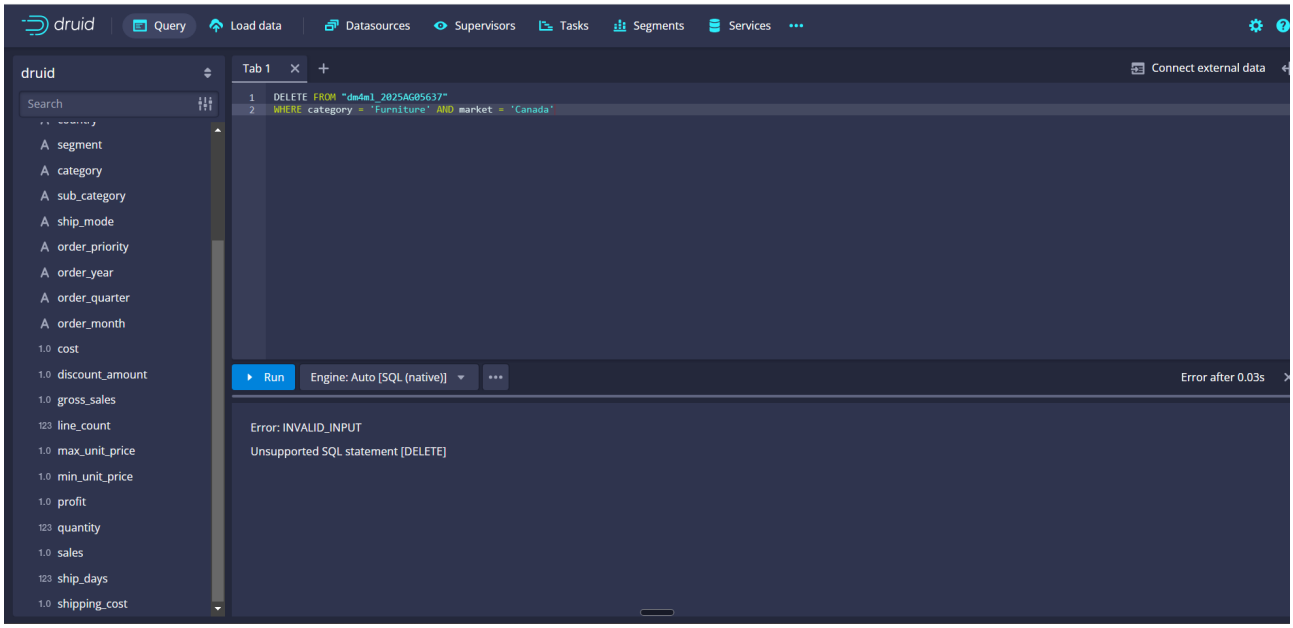


Figure: Query view: response to DELETE

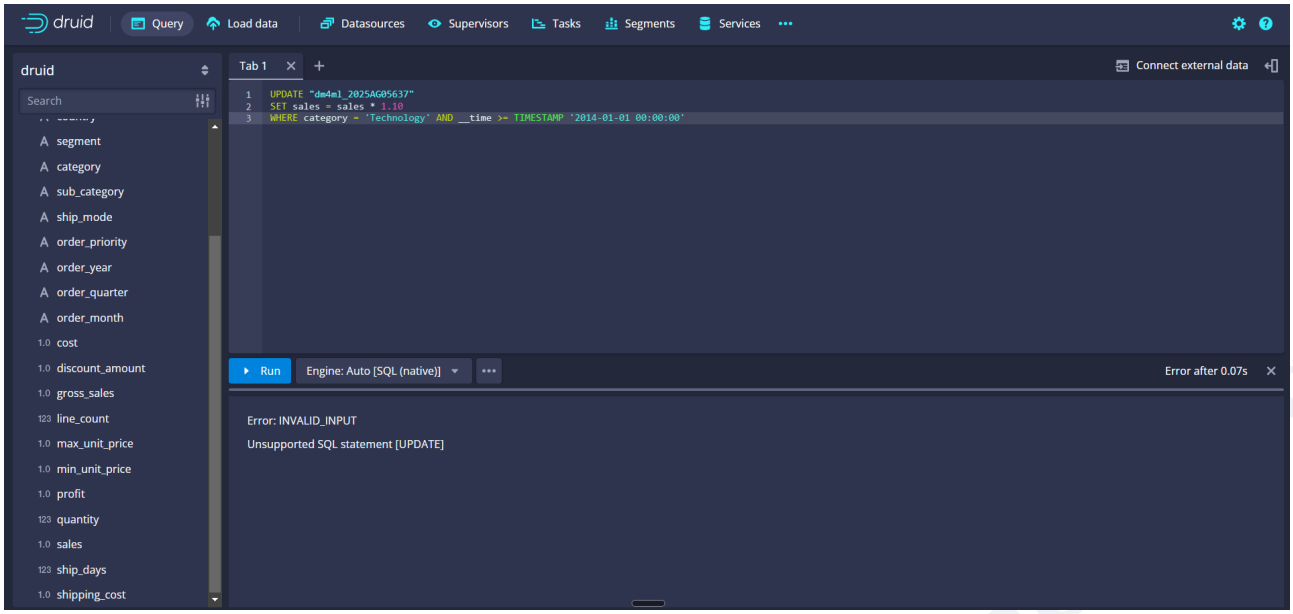


Figure: Query view: response to UPDATE

Why Druid does not support row-level DELETE or UPDATE (~100 words)

Apache Druid is an append-only analytical store. Ingested data is written into immutable, time-partitioned segments that are compressed column by column, indexed with bitmaps and served by historical nodes. Because a segment is never modified in place, Druid has no row-level locking, transaction log or in-place write path, so SQL DELETE and UPDATE are rejected. Ingestion-time rollup also merges many raw rows into one pre-aggregated row, so individual source records may no longer exist to change. These choices make scans, filters and aggregations over millions of rows very fast (OLAP), but make frequent single-row inserts, updates and deletes (OLTP) impractical; changes require re-ingesting whole segments.

(105 words.) Druid's supported way to change data is at segment level: re-ingest or overwrite a time interval (REPLACE ... OVERWRITE, appendToExisting=false), or mark segments unused and run a kill task - whole-interval operations, not transactions on individual rows.

5. OLAP operations with Druid SQL

All queries are saved, with comments, in `olap_queries.sql`. Because rows were rolled up to day granularity at ingestion, the number of original order lines is `SUM(line_count)`.

Q1 - ROLL UP

Day-level data aggregated to month, dropping category and sub-category (market kept).

```
SELECT
  TIME_FLOOR(__time, 'P1M')                    AS order_month,
  market,
  SUM(line_count)                              AS order_lines,
  SUM(quantity)                                AS units_sold,
  ROUND(SUM(sales), 2)                         AS total_sales,
  ROUND(SUM(profit), 2)                        AS total_profit,
  ROUND(SUM(profit) / SUM(sales) * 100, 2)     AS profit_margin_pct
FROM "dm4ml_2025AG05637"
GROUP BY TIME_FLOOR(__time, 'P1M'), market
ORDER BY order_month, market
```

Result: 335 rows (first rows shown).

order_month	market	order_lines	units_sold	total_sales	total_profit	profit_margin_pct
2011-01-01T00:00:00.000Z	APAC	87	319	17616.0	475.39	2.7
2011-01-01T00:00:00.000Z	Africa	63	165	11744.0	1551.49	13.21
2011-01-01T00:00:00.000Z	Canada	2	2	330.0	8.82	2.67
2011-01-01T00:00:00.000Z	EMEA	46	104	7822.0	-816.46	-10.44
2011-01-01T00:00:00.000Z	EU	88	321	16725.0	384.89	2.3
2011-01-01T00:00:00.000Z	LATAM	63	253	21834.0	3021.39	13.84
2011-01-01T00:00:00.000Z	US	76	279	13270.0	2223.8	16.76
2011-02-01T00:00:00.000Z	APAC	96	350	29939.0	2340.97	7.82
2011-02-01T00:00:00.000Z	Africa	51	146	8915.0	1771.68	19.87
2011-02-01T00:00:00.000Z	Canada	5	5	498.0	111.3	22.35

… 325 more rows (full output in results/)

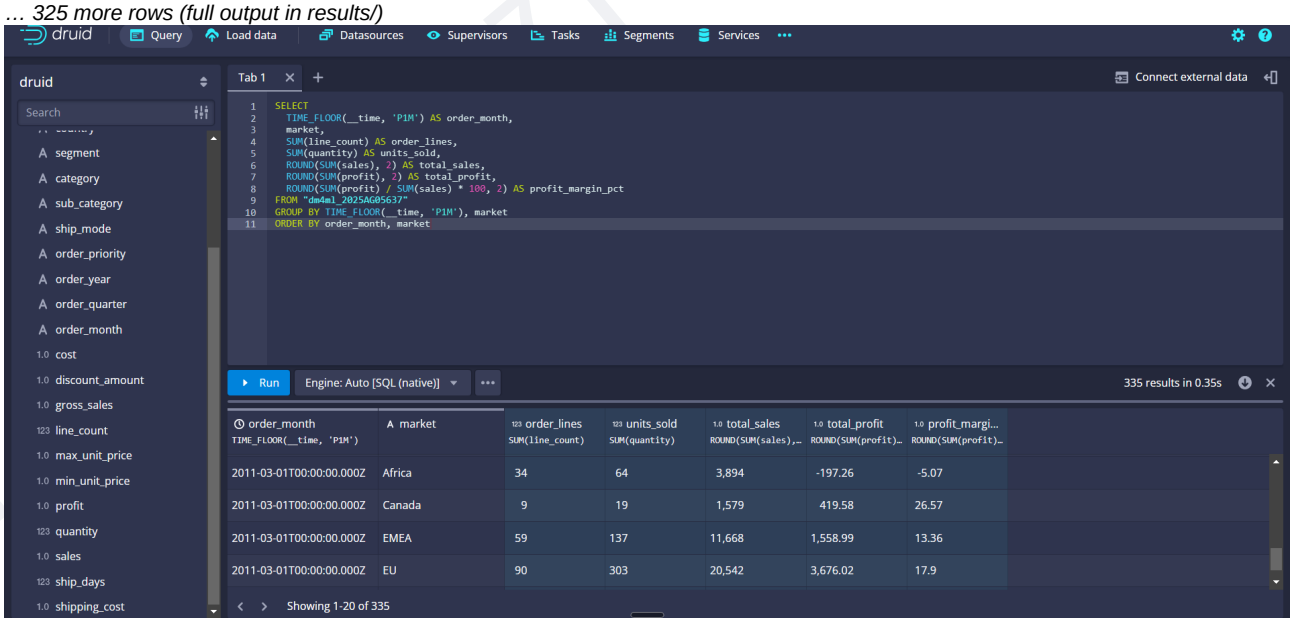


Figure: Q1 ROLL UP result

Q2a - DRILL-DOWN (summary)

Month x category totals for 2014.

```
SELECT
  TIME_FLOOR(__time, 'P1M')                    AS order_month,
  category,
  ROUND(SUM(sales), 2)                         AS total_sales,
  ROUND(SUM(profit), 2)                        AS total_profit
FROM "dm4ml_2025AG05637"
```

```
WHERE __time >= TIMESTAMP '2014-01-01 00:00:00' AND __time < TIMESTAMP '2015-01-01 00:00:00'
GROUP BY TIME_FLOOR(__time, 'P1M'), category
ORDER BY order_month, category
```

Result: 36 rows (first rows shown).

order_month	category	total_sales	total_profit
2014-01-01T00:00:00.000Z	Furniture	58752.0	2596.88
2014-01-01T00:00:00.000Z	Office Supplies	62588.0	7546.69
2014-01-01T00:00:00.000Z	Technology	83061.0	11197.16
2014-02-01T00:00:00.000Z	Furniture	54944.0	3074.64
2014-02-01T00:00:00.000Z	Office Supplies	55194.0	7191.92
2014-02-01T00:00:00.000Z	Technology	62610.0	6598.77
2014-03-01T00:00:00.000Z	Furniture	86502.0	9109.01
2014-03-01T00:00:00.000Z	Office Supplies	74261.0	10247.15
2014-03-01T00:00:00.000Z	Technology	73291.0	8636.81
2014-04-01T00:00:00.000Z	Furniture	68728.0	3196.24

… 26 more rows (full output in results/)

Q2b - DRILL-DOWN (detail)

Technology in December 2014 at day x sub-category level.

```
SELECT
  TIME_FLOOR(__time, 'P1D')                    AS order_day,
  category,
  sub_category,
  SUM(line_count)                              AS order_lines,
  SUM(quantity)                                AS units_sold,
  ROUND(SUM(sales), 2)                         AS total_sales,
  ROUND(SUM(profit), 2)                        AS total_profit
FROM "dm4ml_2025AG05637"
WHERE category = 'Technology'
  AND __time >= TIMESTAMP '2014-12-01 00:00:00' AND __time < TIMESTAMP '2015-01-01 00:00:00'
GROUP BY TIME_FLOOR(__time, 'P1D'), category, sub_category
ORDER BY order_day, sub_category
```

Result: 104 rows (first rows shown).

order_day	category	sub_category	order_lines	units_sold	total_sales	total_profit
2014-12-01T00:00:00.000Z	Technology	Accessories	4	9	611.0	50.09
2014-12-01T00:00:00.000Z	Technology	Copiers	2	10	1978.0	23.34
2014-12-01T00:00:00.000Z	Technology	Machines	5	8	875.0	135.57
2014-12-01T00:00:00.000Z	Technology	Phones	8	32	3022.0	840.89
2014-12-02T00:00:00.000Z	Technology	Accessories	5	37	2430.0	571.91
2014-12-02T00:00:00.000Z	Technology	Copiers	2	8	772.0	248.65
2014-12-02T00:00:00.000Z	Technology	Machines	3	11	1945.0	221.4
2014-12-02T00:00:00.000Z	Technology	Phones	9	33	2141.0	78.88
2014-12-03T00:00:00.000Z	Technology	Accessories	4	8	258.0	-62.55
2014-12-03T00:00:00.000Z	Technology	Copiers	4	13	2364.0	417.65

… 94 more rows (full output in results/)

Roll No: 2025AG05637 | DM4ML Lab Assignm

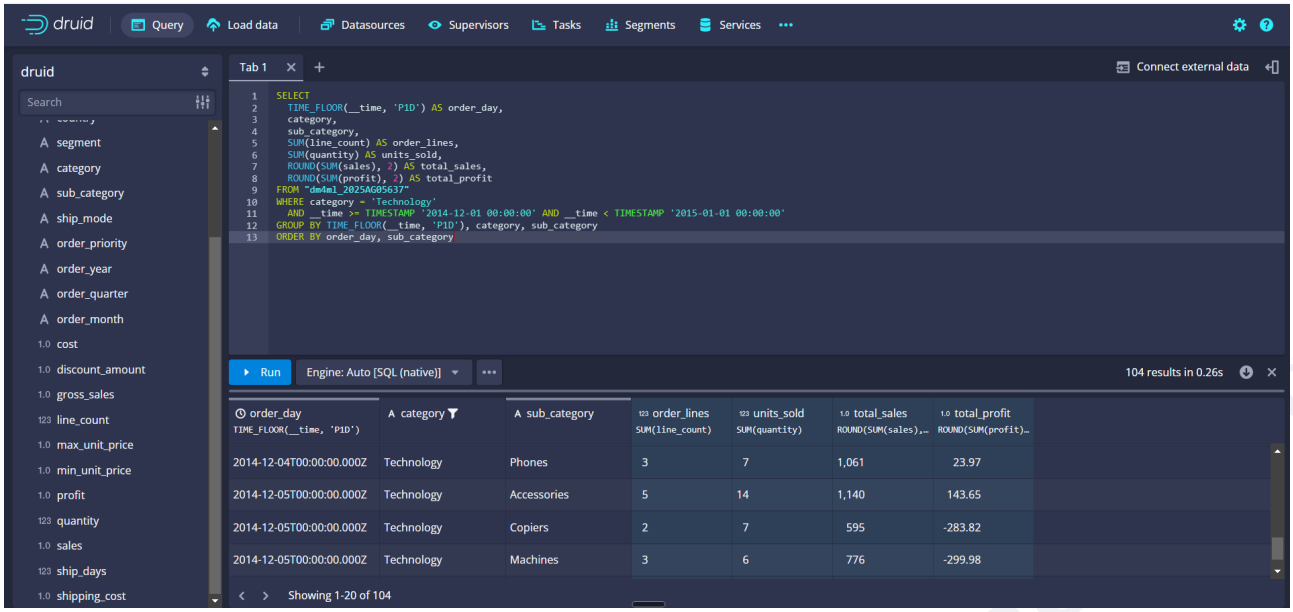


Figure: Q2 DRILL-DOWN result

Q3 - SLICE

Single-dimension filter category = 'Furniture', all years.

```
SELECT
  TIME_FLOOR(__time, 'P1Y')                    AS order_year,
  market,
  SUM(line_count)                              AS order_lines,
  ROUND(SUM(sales), 2)                         AS total_sales,
  ROUND(SUM(profit), 2)                        AS total_profit,
  ROUND(SUM(discount_amount), 2)               AS total_discount_given
FROM "dm4ml_2025AG05637"
WHERE category = 'Furniture'
GROUP BY TIME_FLOOR(__time, 'P1Y'), market
ORDER BY order_year, total_sales DESC
```

Result: 28 rows (first rows shown).

order_year	market	order_lines	total_sales	total_profit	total_discount_given
2011-01-01T00:00:00.000Z	APAC	414	211980.0	14516.25	44736.33
2011-01-01T00:00:00.000Z	US	418	152111.0	5154.02	37281.37
2011-01-01T00:00:00.000Z	LATAM	384	142551.0	7170.89	41263.25
2011-01-01T00:00:00.000Z	EU	260	136836.0	15697.72	31152.3
2011-01-01T00:00:00.000Z	EMEA	149	42820.0	1134.6	13584.17
2011-01-01T00:00:00.000Z	Africa	106	29706.0	1324.75	7140.0
2011-01-01T00:00:00.000Z	Canada	5	481.0	202.11	0.0
2012-01-01T00:00:00.000Z	APAC	471	242525.0	17695.06	47139.34
2012-01-01T00:00:00.000Z	LATAM	515	164737.0	12687.32	38245.5
2012-01-01T00:00:00.000Z	EU	323	163612.0	9244.9	43818.33

… 18 more rows (full output in results/)

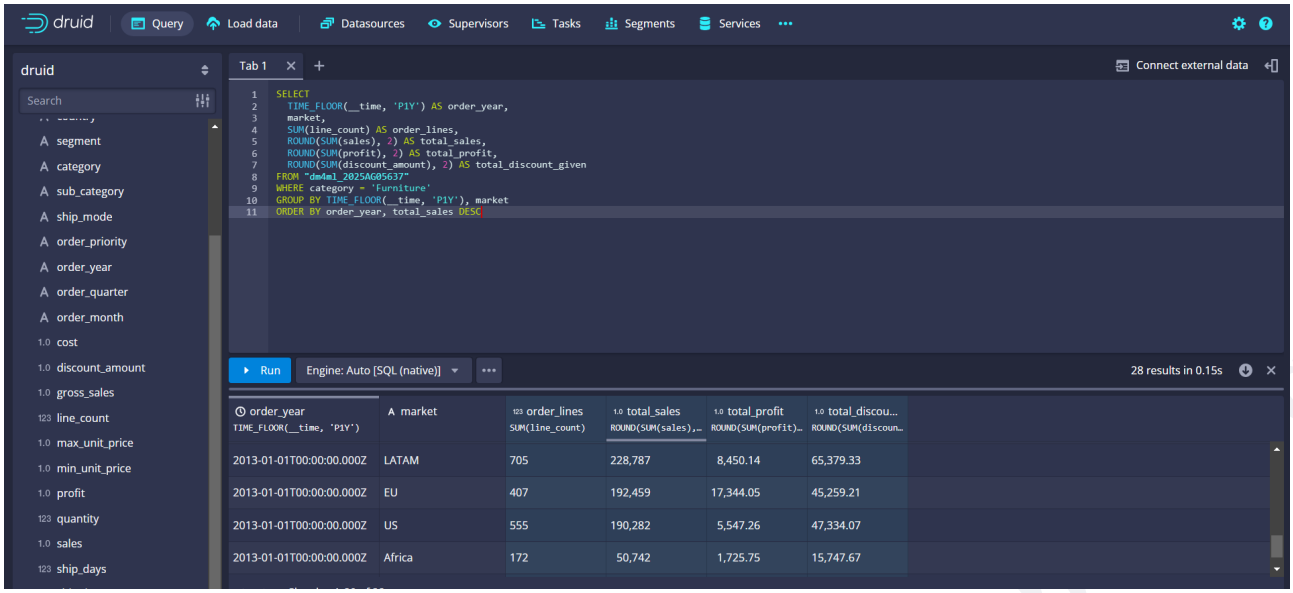


Figure: Q3 SLICE result

Q4 - DICE

category = Technology, market in (APAC, EU), segment = Consumer, year 2013.

```
SELECT
  market,
  sub_category,
  order_quarter,
  SUM(line_count)                                  AS order_lines,
  SUM(quantity)                                    AS units_sold,
  ROUND(SUM(sales), 2)                             AS total_sales,
  ROUND(SUM(profit), 2)                            AS total_profit,
  ROUND(SUM(ship_days) * 1.0 / SUM(line_count), 2) AS avg_ship_days
FROM "dm4ml_2025AG05637"
WHERE category = 'Technology'
  AND market IN ('APAC', 'EU')
  AND segment = 'Consumer'
  AND __time >= TIMESTAMP '2013-01-01 00:00:00' AND __time < TIMESTAMP '2014-01-01 00:00:00'
GROUP BY market, sub_category, order_quarter
ORDER BY market, sub_category, order_quarter
```

Result: 32 rows (first rows shown).

market	sub_category	order_quarter	order_lines	units_sold	total_sales	total_profit	avg_ship_days
APAC	Accessories	Q1	19	66	5935.0	540.69	3.47
APAC	Accessories	Q2	20	78	3163.0	-592.86	4.5
APAC	Accessories	Q3	25	93	5721.0	-103.15	4.12
APAC	Accessories	Q4	31	126	7961.0	740.86	4.13
APAC	Copiers	Q1	10	46	8175.0	459.35	4.3
APAC	Copiers	Q2	26	94	22193.0	5202.59	4.04
APAC	Copiers	Q3	19	73	14035.0	2369.88	3.95
APAC	Copiers	Q4	29	103	21629.0	1659.06	4.1
APAC	Machines	Q1	8	25	2984.0	806.79	2.88
APAC	Machines	Q2	11	60	10367.0	-352.39	4.18

… 22 more rows (full output in results/)

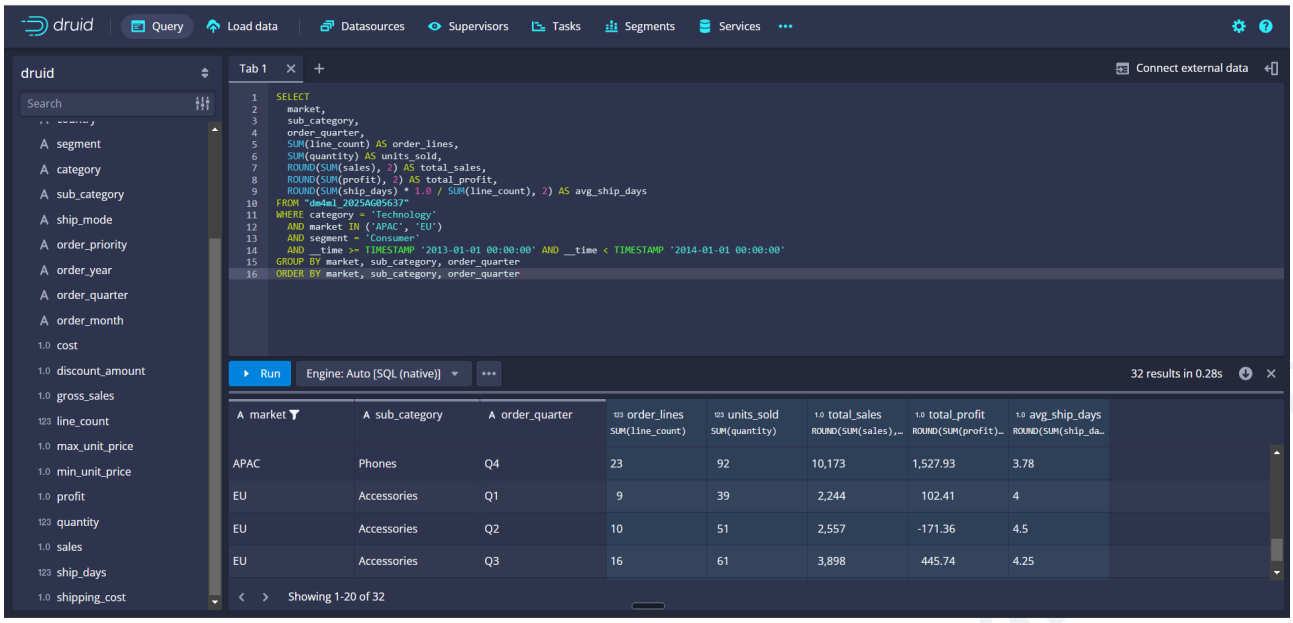


Figure: Q4 DICE result

Q5 - PIVOT

Category rows with quarter columns using SUM(...) FILTER (WHERE ...).

```
SELECT
  category,
  ROUND(SUM(sales) FILTER (WHERE TIME_EXTRACT(__time, 'QUARTER') = 1), 2) AS q1_sales,
  ROUND(SUM(sales) FILTER (WHERE TIME_EXTRACT(__time, 'QUARTER') = 2), 2) AS q2_sales,
  ROUND(SUM(sales) FILTER (WHERE TIME_EXTRACT(__time, 'QUARTER') = 3), 2) AS q3_sales,
  ROUND(SUM(sales) FILTER (WHERE TIME_EXTRACT(__time, 'QUARTER') = 4), 2) AS q4_sales,
  ROUND(SUM(sales), 2)                                                    AS total_sales
FROM "dm4ml_2025AG05637"
GROUP BY category
ORDER BY total_sales DESC
```

Result: 3 rows (first rows shown).

category	q1_sales	q2_sales	q3_sales	q4_sales	total_sales
Furniture	617603.0	846086.0	1050017.0	1359525.0	3873231.0
Technology	568822.0	884950.0	1057824.0	1285632.0	3797228.0
Office Supplies	561162.0	841248.0	993075.0	1165846.0	3561331.0

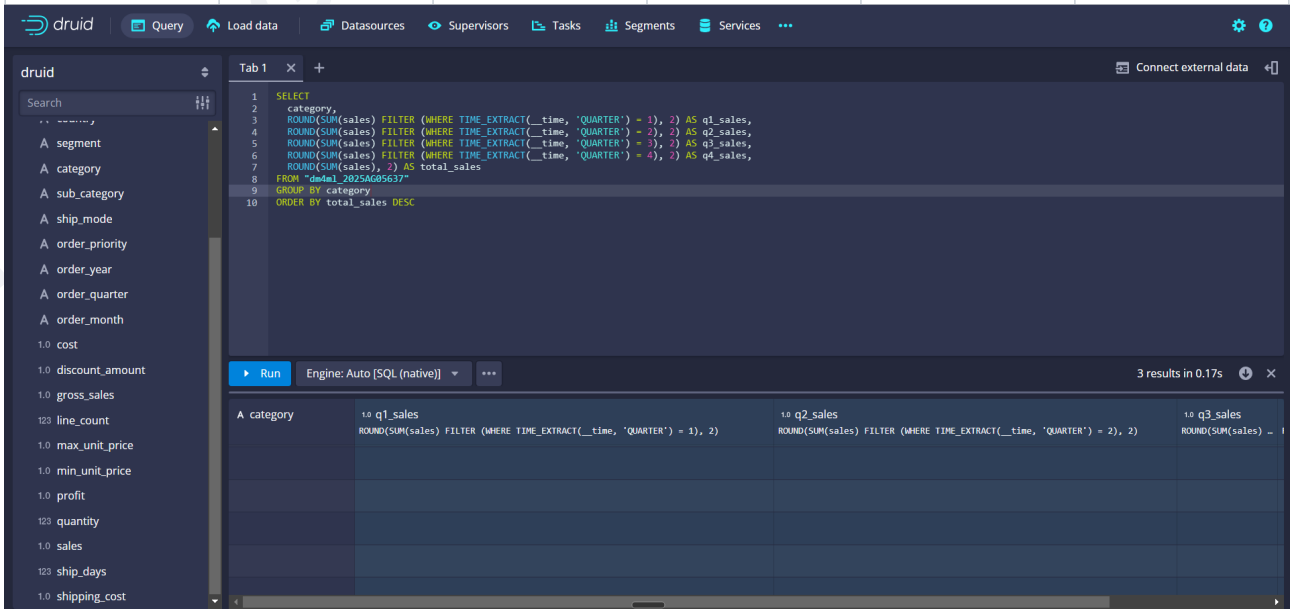


Figure: Q5 PIVOT result

6. Observations

• Preprocessing reduced 51,290 raw rows to 50,425 clean rows covering 2011-01-01 to 2014-12-31.

• Ingestion-time rollup stored 46,631 Druid rows representing 50,425 order lines (about 1.08 input rows per stored row).

• Pivot: Furniture has the highest total sales (3873231.0); its strongest quarter is Q4.

• Roll-up to month x market returned 335 rows - far fewer than the daily x market x category detail, which is the point of the roll-up.

• Both DELETE and UPDATE were rejected by Druid SQL and the data was unchanged, confirming its append-only (OLAP) design.

• Time-based OLAP operations (roll-up, drill-down) map directly onto Druid's __time column with TIME_FLOOR, and segment granularity (month) lets Druid prune whole segments for time-range filters such as the dice query.

No: 2025AG05637 | DM4ML Lab Assignm